

Langage Python

ISUP, Sorbonne Université

Etienne Guével Ingénieur de Recherche - SCAI etienne.guevel@sorbonne-universite.fr
Septembre-Novembre 2025

Partie 4

Packaging, tests unitaires et qualité de code

Rappels

Structurer un programme, version simple

Structure en plusieurs fichiers

- Un ou plusieurs modules contenant les fonctionnalités implémentées
- Un programme principal

```
my_project/  
- module1.py  
- module2.py  
- main.py
```

Structurer un programme, version simple

Cas 1 : un seul fichier avec l'instruction `if __name__ == "__main__":`

```
import math
import sys

def prod(a, b):
    return a * b

def add(a, b):
    return a + b

if __name__ == "__main__":
    print(add(1, 2))
    print(prod(1, 2))
```

Structurer un programme, version simple

Cas 2 : deux fichiers

module1.py

```
import math
import sys

def prod(a, b):
    return a * b

def add(a, b):
    return a + b
```

main.py

```
from module1 import add, prod
print(add(1, 2))
print(prod(1, 2))
```

Structurer un programme, version simple

Problème !

Si les fichiers `main.py` et `module1.py` sont dans le même dossier, vous pouvez faire

```
python main.py
```

Sinon, ça ne fonctionnera pas !

Packaging

Qu'est ce qu'un package ?

Un package python est

1. un **ensemble de modules** organisés en dossiers et sous-dossiers
2. un moyen de créer des libraries spécifiques à un **domaine particulier**
3. **installable** via pip ou conda
4. un moyen d'utiliser les fonctionnalités dans **un autre package**

Par exemple :

- SciPy : dédié au calcul scientifique
- NumPy : dédié au calcul numérique et matriciel
- Pandas : manipulation de tableaux de données et analyses statistiques
- Matplotlib : dédié à la visualisation de données

Exemple

```
package_folder/  
- README.md  
- pyproject.toml  
- mypackage/  
  - __init__.py  
  - mymodule.py  
  - mymodule2.py  
- test/  
  - test_mymodule.py  
  - test_mymodule2.py
```

Installation de la librairie

```
pip install package_folder
```

Utilisation :

```
>>> from mypackage import mymodule  
>>> mymodule.sum(1, 1)  
2
```

Fichier `__init__.py`

- Fichier nécessaire pour importer le dossier en tant que package
- Il est en général vide mais il peut contenir des instructions comme par exemple la version du package
- Plus généralement, il faut aussi créer un fichier `__init__.py` dans chaque sous-dossier.

```
__version__ = "0.0.0"
```

Le fichier README.md est un fichier de documentation, écrit en langage “Markdown”, comme dans les Notebooks Jupyter.

Il doit décrire précisément les fonctionnalités implémentées dans le packages

Fichier pyproject.toml

Le fichier `pyproject.toml` permet de configurer et d'installer le package. Il est écrit en blocs permettant de le configurer selon les spécificités du projet.

```
[project]
# Nom du package pour l'appeler
name = "cours_python"
# Version du package -> utile pour projets longs
version = "0.1.0"
authors = [
    {name = "Etienne Guevel", email="etienne.guevel@sorbonne-
      universite.fr"},
]
description = "Package exemple pour le cours python-ISUP 2025"
# Versions de python compatibles
requires-python = ">=3.10"
# Liste des packages necessaires, avec leur version
dependencies = [
    "matplotlib>=3.10.7",
    "pandas>=2.3.3",
    "seaborn>=0.13.2",
]
```

Fichier pyproject.toml

Le mode d'installation, et les fichiers à installer peuvent aussi être configurés. Plus d'informations sur les modes d'installation ici :

<https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>

```
[build-system]
requires = ["setuptools >= 77.0.3"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
# On indique les chemins des packages
where = ["."]
# On indique les packages
include = ["cours_python"]
# On peut exclure certains dossiers du package
exclude = []
namespaces = false
```

Installation

Une fois que l'on a rempli le `pyproject.toml` on peut l'installer dans notre python avec `pip` :

```
pip install path/to/my/package # . si on est dans le package
```

L'option `-e` permet d'installer le package en mode éditable : si on modifie le code dans le package, pas besoin de le réinstaller python ira chercher directement dans le code !

```
pip install -e .
```

Tests unitaires

Définition

Un test unitaire est une routine permettant de vérifier le bon fonctionnement d'une fonction du package.

Tests unitaires

Définition

Un test unitaire est une routine permettant de vérifier le bon fonctionnement d'une fonction du package.

Par exemple

```
def generate(number):  
    return np.random.normal(0, 1, number)
```

Définition

Un test unitaire est une routine permettant de vérifier le bon fonctionnement d'une fonction du package.

Par exemple

```
def generate(number):  
    return np.random.normal(0, 1, number)
```

Comment tester la fonction generate ?

- Est-ce que `generate(10)` est un `np.array` ?
- Est-ce que `generate(10)` donne la bonne distribution ?
- Est-ce que `len(generate(10))` est effectivement 10 ?

Implémenter des tests unitaires

Les clés pour implémenter des tests

1. Les fonctions de test sont placées dans un dossier test/

Implémenter des tests unitaires

Les clés pour implémenter des tests

1. Les fonctions de test sont placées dans un dossier test/
2. Les fonctions de test utilisent le mot clé “assert”

```
from mypackage.mymodule import generate
def test_generate():
    assert len(generate(10)) == 10
```

Implémenter des tests unitaires

Les clés pour implémenter des tests

1. Les fonctions de test sont placées dans un dossier test/
2. Les fonctions de test utilisent le mot clé “assert”

```
from mypackage.mymodule import generate
def test_generate():
    assert len(generate(10)) == 10
```

3. Conventions pour les noms des fonctions de test

Implémenter des tests unitaires

Les clés pour implémenter des tests

1. Les fonctions de test sont placées dans un dossier test/
2. Les fonctions de test utilisent le mot clé “assert”

```
from mypackage.mymodule import generate
def test_generate():
    assert len(generate(10)) == 10
```

3. Conventions pour les noms des fonctions de test
 - Un **module de test** par module de package.
Ex : pour **tester le module** mymodule.py, le nom du fichier de test sera test_mymodule.py

Implémenter des tests unitaires

Les clés pour implémenter des tests

1. Les fonctions de test sont placées dans un dossier test/
2. Les fonctions de test utilisent le mot clé “assert”

```
from mypackage.mymodule import generate
def test_generate():
    assert len(generate(10)) == 10
```

3. Conventions pour les noms des fonctions de test
 - Un **module de test** par module de package.
Ex : pour **tester le module** mymodule.py, le nom du fichier de test sera test_mymodule.py
 - Une **fonction de test** par fonction testée.
Ex : pour **tester la fonction** generate, le nom de la fonction de test sera test_generate

Exemple

Pour tester la fonction

```
def hello(name):  
    return f"Hello {name}"
```

le test est

```
def test_hello():  
    assert hello("python") == "Hello python"
```

Pour tester cette fonction on peut utiliser le package python `pytest`, qui permet d'exécuter tous les tests présents dans un dossier / fichier, et de résumer leurs résultats.

Exemple

Pour tester la fonction

```
def hello(name):  
    # Une erreur a ete introduite ici  
    return f"Hello name"
```

le test est

```
def test_hello():  
    assert hello("python") == "Hello python"
```

```
test.py F [100%]  
  
===== FAILURES =====  
_____ test_hello _____  
  
def test_hello():  
> assert hello("Etienne") == " Hello Etienne "  
E     AssertionError: assert ' Hello name ' == ' Hello Etienne '  
E  
E     - Hello Etienne  
E     + Hello name  
  
test.py:5: AssertionError  
===== short test summary info =====  
FAILED test.py::test_hello - AssertionError: assert ' Hello name ' == ' Hello Etienne '  
===== 1 failed in 0.02s =====
```

Quelques conseils

Il est important

- d'implémenter des modules python **par thème**

Il est important

- d'implémenter des modules python **par thème**
- d'implémenter une fonction par “fonctionnalité” et éviter du code trop lourd

Il est important

- d'implémenter des modules python **par thème**
- d'implémenter une fonction par “fonctionnalité” et éviter du code trop lourd
- de proscrire la **duplication de code**

Il est important

- d'implémenter des modules python **par thème**
- d'implémenter une fonction par “fonctionnalité” et éviter du code trop lourd
- de proscrire la **duplication de code**
- de tester chaque fonctionnalité en un **minimum de code**

Outils de tests unitaires : Pytest

Deux packages à connaître

- pytest : à la racine du projet, exécuter dans un terminal

```
python -m pytest -v
```

Deux packages à connaître

- `pytest` : à la racine du projet, exécuter dans un terminal

```
python -m pytest -v
```

- `pytest-cov` : un plugin pour mesurer la couverture des tests

```
python -m pytest -v --cov
```


Outils de tests unitaires : Pytest

Deux packages à connaître

- `pytest` : à la racine du projet, exécuter dans un terminal

```
python -m pytest -v
```

- `pytest-cov` : un plugin pour mesurer la couverture des tests

```
python -m pytest -v --cov
```

Le package `unittest` est également utilisé pour implémenter des tests unitaires avec une syntaxe différente de `pytest`.

A vous de jouer

Dans votre package

- Implémentez des tests unitaires pour **chacune des fonctions**
- Exécutez les tests avec `pytest`

Qualité de code

PEP8 – Style de code

Les Python Enhancement Proposals, ou PEP, sont un ensemble de recommandations liées au langage python.

La PEP8 définit des conventions **assez restrictives** de style de code.

Page officielle : peps.python.org/pep-0008

Longueur des lignes

La PEP8 recommande entre 80 et 100 caractères maximum -> cela permet d'ouvrir deux fichiers sur une même fenêtre et de voir tous le code dessus.

Conventions PEP8 à retenir

Indentation

Utiliser une indentation à **4 espaces exactement** et ne pas utiliser de **tabulation**.

Exemple pour un appel de fonction

```
foo = long_function_name(  
    var_one, var_two, var_three, var_four  
)
```

ou bien

```
foo = long_function_name(  
    var_one,  
    var_two,  
    var_three,  
    var_four  
)
```

Conventions PEP8 à retenir

Blocs de code et retours à la ligne

Coder des blocs de code **cohérents**

Séparés les blocs de code par un **unique** retour à la ligne.

Encoding et langue

Utiliser l'anglais et proscrire les caractères spéciaux (encodage UTF-8)

Conventions PEP8 à retenir

Imports

- Un import par ligne
- Préférer les imports **absolus**
- Ordre des imports : package de la librairie standard, librairies installées puis librairies locales

Par exemple :

```
import os
import sys
import time

import numpy as np

import mypackage
```


Chaîne de caractères

Il est possible de définir des chaînes de caractères avec " " ou bien ' '.

PEP8 ne donne pas de recommandation mais il est important de ne pas utiliser les deux façons de faire (uniformisation du code)

Conventions PEP8 à retenir

“Whitespaces” dans les expressions

Proscrire les ‘espaces blancs’ inutiles dans les situations suivantes

Correct :

```
spam(ham[1], {eggs: 2})
foo = (0,)
if x == 4: print(x, y); x, y = y, x
spam(1)
dct['key'] = lst[index]
```

Incorrect :

```
spam( ham[ 1 ], { eggs: 2 } )
bar = (0, )
if x == 4 : print(x , y) ; x , y = y , x
spam (1)
dct ['key'] = lst [index]
```

Conventions PEP8 à retenir

“Whitespaces” dans les expressions (suite)

Proscrire les ‘espaces blancs’ inutiles dans les situations suivantes

Correct :

```
x = 1
y = 2
long_variable = 3
```

Incorrect :

```
x           = 1
y           = 2
long_variable = 3
```

Conventions PEP8 à retenir

“Whitespaces” dans les expressions (suite)

Proscrire les ‘espaces blancs’ inutiles dans les situations suivantes

Correct :

```
def complex(real, imag=0.0):  
    return magic(r=real, i=imag)
```

Incorrect :

```
def complex(real, imag = 0.0):  
    return magic(r = real, i = imag)
```

Conventions PEP8 à retenir

“Whitespaces” dans les expressions (suite)

MAIS ne pas supprimer tous les espaces blancs, par souci de lisibilité !

Correct :

```
i = i + 1
submitted += 1
x = x*2 - 1
hypot2 = x*x + y*y
c = (a+b) * (a-b)
```

Incorrect :

```
i=i+1
submitted +=1
x = x * 2 - 1
hypot2 = x * x + y * y
c = (a + b) * (a - b)
```

Conventions PEP8 à retenir

Docstrings

Les docstrings sont des commentaires insérés dans le code, en début de modules ou de fonctions. Ils résument ce que fait la fonction et indiquent quels sont les arguments et les valeurs de retour.

```
def sample(n):  
    """  
    Get a sample from Gaussian distribution  
  
    Args:  
        n: the number of desired normal values  
  
    Returns  
        A sequence of n Gaussian values  
    """  
    return normal(0, 1, number)
```

Voir aussi la PEP257 : peps.python.org/pep-0257

Conventions de nommage

Il existe différentes façon de nommer en python :

- lowercase ou snake_case
- lowercase_with_underscores
- UPPERCASE
- UPPERCASE_WITH_UNDERSCORES
- CapitalizedWords

Conventions de nommage

Les conventions de nommage sont les suivantes :

- Packages : court, lowercase
- Modules : court, lowercase ou lowercase_with_underscores
- Classes : CapitalizedWords
- Variables locales : lowercase ou lowercase_with_underscores
- Variables globales : UPPERCASE ou UPPERCASE_WITH_UNDERSCORES

Analyse de code

`pylint` est un outil permettant de mesurer la qualité de code selon les conventions de la PEP8 et met une note sur 10

A la racine de votre package

```
python -m pylint mypackage
```

ou simplement

```
python -m pylint mymodule.py
```

A vous de jouer

1. Placez-vous à la racine de votre package et exécutez pylint

```
python -m pylint mypackage
```

2. Quelle est votre note sur 10 ?
3. Améliorez votre code et exécutez à nouveau pylint